

FluxFraud: mathematics to code

Implementation companion to the mathematical foundations

This companion maps the 41-page mathematical guide to the current project. It separates the model specification from implementation details so that the main guide can be read as mathematical notes. File paths below are relative to the project root; line numbers refer to the inspected source snapshot and can change after edits.

Read the execution path first. A configuration enters the simulator, producing accounts, attempted transactions and hourly summary states. A feature builder walks the transaction history in order. Evaluation fits models on eligible earlier data and produces test predictions and fixed policies. The application converts the resulting history into an animated replay.

The main entry points are **src/fluxfraud/simulator.py:19**, **features.py:18**, **evaluation.py:67** and **playback.py:52**. The reusable stochastic primitives live in **processes.py**. **config.py** defines exposed parameters and validation. **cli.py** connects the pipeline for saved experiments, while **app.py** supplies the interactive interface.

What is persisted. A command-line run exports the exact configuration, a runtime manifest, metrics, policies and a SQLite database. The database contains transactions, accounts, latent summaries, features and predictions. Hidden fields are retained for research inspection; their presence in the database does not make them model inputs.

Scope of this document. The following pages cover generation order, parameter ownership, feature construction, estimator training, metric interpretation and reproducible use. This is a snapshot of the existing implementation, not a claim that new model mechanisms or diagnostics were added while producing the PDFs.

The mathematical guide includes the integrated-intensity diagnostic and event-time likelihood as possible extensions. The code does not currently fit Hawkes parameters, trace fund provenance, simulate cross-account excitation, perform online retraining or model detector-driven blocking. Keeping those boundaries explicit prevents theory examples from being mistaken for implemented features.

18 September 2026

1. Population and hourly update mapping

Main guide pages 3-9: population and discrete states. In `simulator.py:20-40`, three random streams are created, followed by community, baseline log scale, activity, ring and opening balances. The ring contains $\max(3, N//20)$ distinct accounts. All behavioural states begin ordinary, all compromise indicators begin false, and excitation and pair counts begin at zero.

Main guide pages 10-13: spending dynamics. `processes.py:6` defines `ou_step`. It checks finite inputs, positive mean-reversion rate, nonnegative diffusion and nonnegative step size. The expression using `expm1` evaluates one minus an exponential accurately when its argument is small. It is algebraically the same exact transition derived in the guide.

Hourly order in `simulator.py:41-54`. The code first computes the environmental shift, updates behaviour, samples recovery and infection transitions based on the previous compromise state, computes the spending target, advances `x` by one hour, and constructs the background rate. Only then are that hour's event times generated.

A detail worth checking. The variable `log_mean` is the fixed baseline `m` with superscript zero; `x` is the evolving log-spending state. Neither is a bank balance. The account table stores latent activity and ring membership, but the transaction feature builder is not allowed to consume them.

Example. Setting `ou_sigma` to zero removes stochastic innovations from the log-spending update. It does not freeze spending: changing behavioural targets still move `x`, and individual amounts still include event-level Gaussian noise with standard deviation 0.60.

The behavioural transition matrix, community count, amount-noise scale, fraud mark probability, opening-balance range and several multipliers are currently constants inside the simulator. They are not exposed configuration fields. Altering them means changing the model specification, not merely choosing a different seed.

Equation correspondence. Guide equations (7.1)-(7.3) map to behaviour sampling; (9.1)-(9.3) map to compromise and fraud labelling; (11.3) maps to `ou_step`; (13.1) defines the hourly spending target. Page numbers are also the equation-number prefixes.

2. Event times, marks and ledger

Main guide pages 14-19. `processes.py:20` defines `hawkes_interval`. It samples candidate waits under the current upper bound, decays excitation, retains candidates according to the intensity ratio, and adds eta times beta only for retained events. It returns both event offsets and the residual at the interval endpoint.

`simulator.py:54-65` generates one hour of event times per account, carries the returned excitation into the next hour, merges all pending events and sorts them. Generating clocks account by account is valid here because neither recipients nor balances feed back into within-hour event rates.

Main guide pages 20-25. At each sorted event, `simulator.py:68-87` samples the fraud label, constructs candidates excluding the sender, calculates recipient energies, samples the receiver, and then samples amount and binary marks. `processes.py:53` defines stable Gibbs normalisation by subtracting the minimum cost before exponentiating.

Main guide page 26. The balance check and transfer update occur in `simulator.py:88-93`. The acceptance field means enough sender funds were available. A false value does not mean the recipient declined or a detector intervened. Pair memory increments for every attempt, regardless of settlement.

Example. An attempt of 10,000 minor units from an account holding 9,000 remains in the transaction table with `accepted = 0`. Its amount and true fraud label are retained, and later temporal and pair-count features see it. Neither account balance changes. A discarded thinning candidate creates no row at all.

The descriptive `typology` field calls a fraudulent event from a sender inside the hidden set a mule transfer; otherwise fraudulent events are account takeover. This is a synthetic naming convention. It is not a transaction-level inference made by a trained model.

Safety limits. The event cap raises an error rather than silently truncating a successful result. Nonfinite rates and unresolvable time increments also raise errors. These checks prevent some invalid outputs, but they do not make every extreme parameter setting computationally practical.

3. Configuration and units

Exposed defaults in config.py. The command-line configuration begins with 160 customers, 45 days and seed 42. The application initially uses 64 customers and 30 days. A horizon is always days times 24 hours. Use the configuration saved with a result when interpreting its timestamps.

Event clock. **base_rate = 0.10** is per customer per hour before multipliers. **branching_ratio = 0.35** is dimensionless integrated excitation. **decay = 1.5** has units inverse hours. The configuration restricts the branching ratio to $[0, 0.95)$, a stricter operational range than the mathematical subcritical boundary of one.

Spending dynamics. **ou_kappa = 0.06** has units inverse hours; **ou_sigma = 0.12** has units inverse square-root hours. **temperature = 0.8** and **network_memory = 1.0** are dimensionless preference parameters. Their ratio controls the power on cumulative pair count.

Latent compromise. **compromise_hazard = 0.0007** and **recovery_hazard = 0.10** are per-hour hazard parameters converted to hourly probabilities. They are not direct event-level fraud probabilities. **drift_fraction = 0.75** and **drift_strength = 1.0** govern the scheduled bundled change.

Experiment policy. **label_delay_hours = 24** controls maturity; **alert_budget = 20** controls daily top-K review and validation threshold targeting. **max_events = 1,000,000** is a guard, not a target sample size. The number of generated events is random.

Example. A 30-day run shifts at hour 540 and starts testing at hour 504. A 45-day run shifts at hour 810 and starts testing at hour 756. Reusing the first run's calendar boundaries to interpret the second run's plots would be wrong even though all rate parameters are unchanged.

Unknown configuration fields, nonfinite values and invalid types fail validation. Customer count is limited to 10-2,000 and days to 10-365. A mathematically valid value can still lead to an expensive run; dense recipient selection examines all other accounts for every attempt.

4. Exact feature definitions

Source: features.py:12-15 and 46-84. There are 14 features. The six transaction features are log amount, sine and cosine of time of day, foreign, new device, and amount divided by max(pre-event balance, 1). Amount and balance use the same minor units for that ratio.

The four temporal features are prior attempt counts in the open intervals (t-1, t) and (t-24, t), the historical transformed-amount deviation, and seconds since the previous sender attempt. The latter is capped at 86,400 seconds and starts at that value for a sender with no history.

The four network features are whether the pair has never appeared, the prior pair count, the receiver's number of distinct prior senders, and the sender's expanding recipient entropy. Counts include declined attempts. Neither the final graph nor future events may enter these features.

Example. An event at time 10.0 does not count an earlier event exactly at 9.0 in its one-hour count because the left boundary is open. An event at 9.0001 does count. The current event is also excluded because all feature values are appended before the historical state is updated.

Online moment update. Welford's recurrence maintains previous count n , mean m and sum of squared deviations M_2 without retaining every transformed amount. For a new value l , the update is:

$$\delta = l - m, \quad m' = m + \frac{\delta}{n+1}, \quad M_2' = M_2 + \delta(l - m'). \quad (5.1)$$

The feature uses the old moments and then applies this update. Sample variance is $M_2/(n-1)$ when at least two prior observations exist. The standard-deviation floor is 0.3, and the deviation feature is zero for insufficient history.

Input safeguards. Required fields must exist, transaction identifiers must be unique, timestamps must be strictly increasing, numeric fields finite and amounts positive. These conditions make causal iteration well defined. Account identifiers select historical records internally but are not columns in the fitted feature matrix.

Main-guide correspondence. Recipient entropy is explained on page 27. The distinction between observable history and hidden generator state appears on pages 29 and 36.

5. Training and validation policy

Source: `evaluation.py`:15-23 and 67-122. The first half of simulated time supplies training events whose labels have matured by the half-way boundary. Validation occupies the next 20% and applies its own maturity cutoff. The final 30% supplies test events. Models are fitted once, before test scoring.

Logistic regression. A pipeline fits **StandardScaler** on training data and then **LogisticRegression(max_iter=1500)** on all 14 features and training labels. Other estimator options use the installed scikit-learn defaults; their exact versions belong in the run manifest.

Boosting variants. Three **HistGradientBoostingClassifier** instances use **max_iter=100**, **max_leaf_nodes=15**, **l2_regularization=2** and the configured seed. They use all 14 features, the six transaction features, and transaction plus temporal features respectively. Other defaults, including automatic early-stopping behaviour for sufficiently large training sets, remain library-controlled.

Isolation Forest. It uses 100 trees, the configured seed and one worker. Fitting receives features only, on the same training-row mask. The training mixture can contain fraud. Scores are the negative of **score_samples**, so larger values mean more anomalous. The behavioural rule is amount deviation + 2 times new-device + foreign, with no fitted coefficients.

Threshold selection. Let V be the effective validation duration in days after the label-maturity exclusion, n the validation count and K the budget. The target flagged fraction is $\min(KV/n, 1)$. Each threshold is the corresponding upper quantile of validation scores, using the higher order-statistic convention.

Example. With $K = 20$, $V = 5$ days and 1,000 validation events, the target fraction is 0.10 and the threshold is the 90th-percentile validation score. Tied scores can produce more flags than the target; a changed test distribution can also change the realised alert rate.

Validation labels are not needed to compute this score quantile, although validation rows still follow the project's maturity filter. No test labels select the threshold. Training requires both classes for the supervised estimators, and every split must contain at least ten observations.

6. Metrics, replay and loops

Source: `evaluation.py`:26-63. Thresholded precision and recall use the fixed validation policy. Daily top-K metrics instead rank all test events within each calendar day and select at most K. This is retrospective batch triage, not a causal online rule that knows when to spend its budget during the day.

Selection is performed once over the whole test set, then sliced into pre-shift and post-shift periods. This prevents a day crossing the shift from receiving two independent budgets. A partial test day still receives the full K allowance.

Example. If a day contains 100 events and $K = 20$, daily selection reviews its 20 highest scores even when a fixed threshold would flag only seven or as many as forty. The two policies can have different precision and recall without either calculation being inconsistent.

Average precision is computed by **`average_precision_score`**; it is not a trapezoidal area calculation. ROC AUC requires both classes. Brier score is reported for supervised probability outputs. Settled-fraud value capture weights positives by amount times acceptance, so declined attempts contribute zero transferred value. This is captured value in a retrospective review set, not money actually blocked by the simulator.

Replay: `playback.py`:52-87. The interface receives a stored event payload with true labels, settlement flags, test-period model flags and selected loops. It advances through that history. Moving the day slider does not refit the models or generate a new stochastic future. Cumulative performance plots describe the displayed test prefix.

Loops: `playback.py`:17-49. Only settled edges are retained for a 24-hour history. A reverse edge closes a two-edge loop; otherwise the search may find an earlier ordered two-step path that the current event closes into a three-edge loop. Edges exactly 24 hours old remain eligible. One representative is recorded per event, with preference for the shorter loop.

The visual node arrangement is a display layout, not physical space or a learned embedding. Ground-truth colours explain what the generator sampled. Detector flags are a separate layer. Neither the geometry nor a highlighted loop alone establishes fraud.

7. Reproducing and extending the experiment

Minimal local workflow. Follow **HOW_TO_USE.txt** to create a Python environment and install the project with its application and development extras. The project supports Python 3.11 or newer. Run the existing tests before changing process equations.

A saved experiment uses **fluxfraud run --config configs/default.json --out runs/demo**. Choose a fresh output directory: the command refuses to overwrite an existing run. Start the interactive replay with **streamlit run app.py**. These commands are instructions, not evidence that a new experiment or deployment was performed for this document.

Monte Carlo. **fluxfraud monte-carlo --config configs/default.json --seeds 11 22 33 44 55 --out runs/mc** evaluates full, no-excitation, no-memory and no-shift configurations. It saves replicate-level metrics and a summary. Main-guide pages 33-34 explain why seeded variation and ablation confounding matter.

Record what changed. Store the complete configuration, code revision, dependency versions and seed list with results. The exported manifest includes versions and a transaction fingerprint. A changed seed explores another realisation; a changed parameter explores another world law; a changed implementation may alter the law unintentionally.

Example extension. Adding memory decay requires more than replacing a displayed number. Define the decay equation, specify whether it applies to attempts or settlements, update recipient choice and any affected features consistently, and add a test that distinguishes decaying from cumulative memory. Then compare matched experiments and report the change in event or graph statistics.

Verification boundaries. The existing tests cover process behaviour, causal features, evaluation conventions and replay logic. The guide's likelihood and time-rescaling discussion does not mean those diagnostics are implemented. Likewise, the PDFs do not establish real-world calibration or production readiness.

Publication. The intended public repository is **yamirafroz-ux/fraud-detection-evaluation**. This document does not assert that a remote repository has been created, pushed or deployed. Publishing the project is separate from producing and reviewing these notes. Keep generated private data, credentials and environment files out of any future public commit.